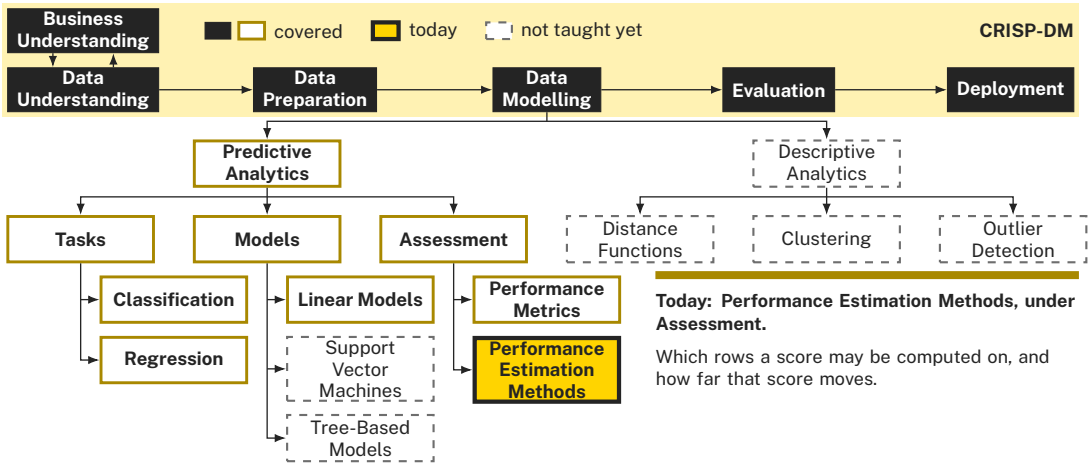
DALHOUSIE UNIVERSITY

Performance Estimation

Holdout, random subsampling, bootstrap, and cross-validation

Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

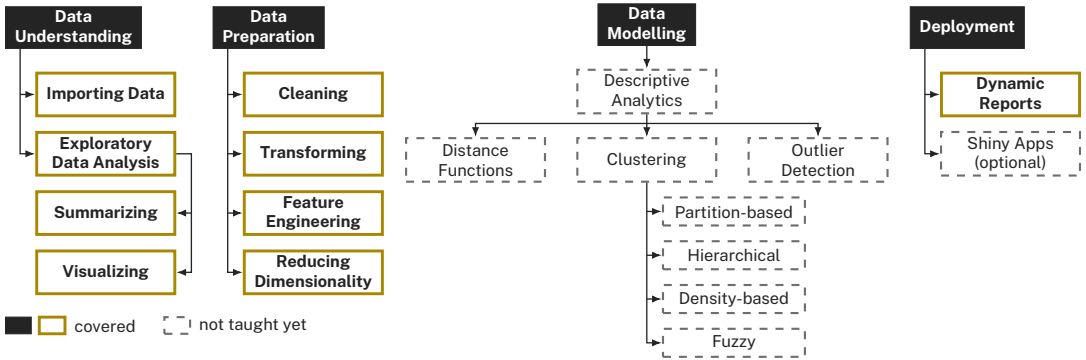
Where performance estimation sits



Today: Performance Estimation Methods, under Assessment.

Which rows a score may be computed on, and how far that score moves.

The rest of the map, for orientation



Performance estimation

DALHOUSIE UNIVERSITY

A score computed on the data that built the model is not an estimate of anything we care about.

Two goals, two questions

DALHOUSIE UNIVERSITY

GOALS

- Help analysts in selecting the best possible prediction model from a large set of available alternatives.
- Provide end-users with reliable estimates of the performance of the models on future or unseen data.

QUESTIONS

- What performance metric to use?
- How to reliably estimate the predictive performance of a data science solution using the available data?

Resubstitution estimates

DALHOUSIE UNIVERSITY

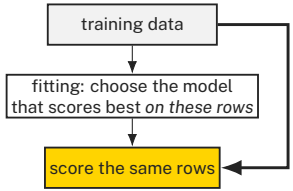
- Resubstitution estimates result from evaluating the model's predictive performance **on the data used to train the model**.
- Resubstitution estimates are **unreliable**.



Why do you think this is?

Resubstitution is unreliable I

- Models try to *optimize* predictive performance on the available data set (e.g., minimize MSE or maximize accuracy).
- We expect they achieved good scores on the training data.
- Resubstitution estimates will be **over-optimistic**!



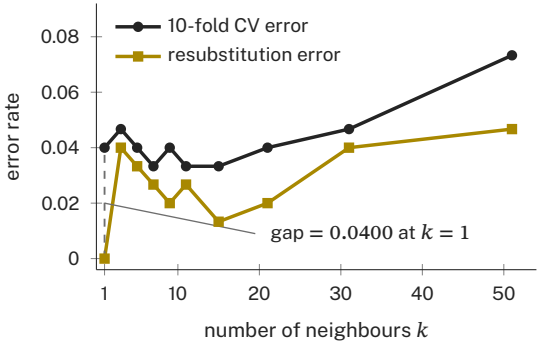
the same rows sit on both ends of the loop, so the optimizer has already seen the answer key

The optimism, made visible

k-nearest-neighbour classifiers on `iris` ($N = 150$), scored two ways.

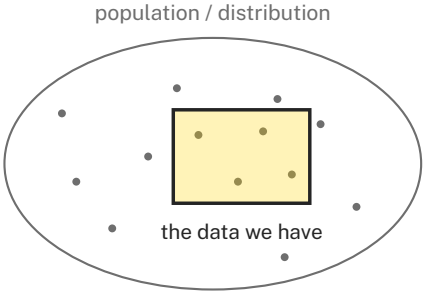
- At $k = 1$ every case is its own nearest neighbour, so the resubstitution error is **exactly zero**.
- The honest error at $k = 1$ is **0.0400**.
- The gap is largest at $k = 1$, so resubstitution picks the wrong k.

Stratified 10-fold CV, seed 3141.



Resubstitution is unreliable II

- The available data set is just a **sample** of the population (i.e., the entire group that we want to make predictions about).
- We would like to know how the model will perform on the whole distribution.
- We usually don't have access to data about the whole population.
- The best we can do is to evaluate the model on **different samples** of this distribution.



every estimate we can make is a resample *within* the gold box

Reliable error estimates I

DALHOUSIE UNIVERSITY

Goal: obtain a reliable estimate of the expected prediction error of a model on the *unknown* data distribution.

- Estimate based on evaluation on unseen cases, a **test set**.
- Ideally, we want to repeat the evaluation process several times.
  - Calculate several scores using different subsamples of our data.
  - Use the average score as our estimate, a more robust estimate of the true population mean prediction error.
  - Have a measure of uncertainty using the standard error of the estimate.

The **Golden Rule** says that data used for evaluating (or comparing) any models should **never** be seen or used during model development.

Reliable error estimates II

Calculate several estimates of error $E_1, E_2, \ldots, E_K$, then a sample mean prediction error:

$$\bar{E} = \frac{1}{K}\sum_{i=1}^{K} E_i$$

and, for independent scores, its standard error:

$$SE(\bar{E}) = \frac{s_E}{\sqrt{K}}, \qquad s_E = \sqrt{\frac{1}{K-1}\sum_{i=1}^{K}\left(E_i - \bar{E}\right)^2}$$

WORKED, WITH $K = 5$

Five fold errors:

$E = (0.0000,\ 0.0333,\ 0.0000,\ 0.0667,\ 0.0667)$

$$\bar{E} = \frac{0.1667}{5} = 0.0333$$

$$s_E = \sqrt{\frac{0.004444}{4}} = 0.0333$$

$$SE(\bar{E}) = \frac{0.0333}{\sqrt{5}} = 0.0149$$

Mean 0.0333, fold SD 0.0333. The naive SE 0.0149 ignores fold dependence.

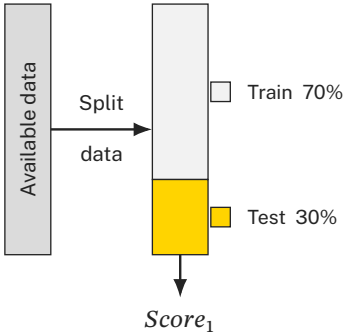
Performance estimation methods

DALHOUSIE UNIVERSITY

Four ways to score a model on rows it never saw.

Holdout | Random subsampling | Bootstrap | Cross-validation

Holdout: one split, one score



1. Split the data into a training and a test set.
   - Common split, 70% training, 30% testing.
2. Train a model using the training set.
3. Make predictions for the test set and calculate the performance score.

The split is made *before* anything is fitted, and the test rows are not looked at again. That is the Golden Rule in its simplest form.

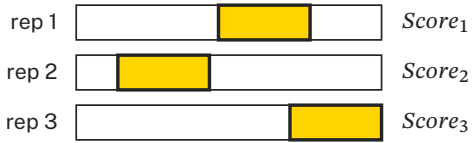
What one split cannot tell you

IF THE DATA SET IS VERY SMALL

- Test set too small: unreliable estimates.
- Training set too small: a worse model than what could be obtained with the available data.
- One split cannot measure **variability across splits**.

SOLUTION: RANDOM SUBSAMPLING

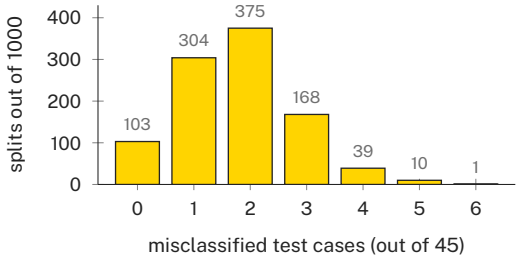
Repeat the random split several times.



test blocks may overlap between repetitions

Average the scores and report how much they vary across splits.

How far one split moves the answer



1000 random 70/30 splits of `iris`, 3-NN, seeds 0–999.

A single holdout is an estimate that may depend strongly on the chosen split.

The model never changes. The *split* changes.

- Mean error **0.0393**, standard deviation **0.0232**.
- Best split reports **0.0000**. Worst split reports **0.1333**.
- Seed 42 gives 0.0000; seed 7 gives 0.0667. Both are “the” holdout estimate.
- Cut the training set to 30 rows and the worst split reports **0.4083**.

Setup for Every Listing

R

```r
library(caret)    # knn3, train, createFolds
library(rsample)  # mc_cv
library(boot)     # boot
```

R ships `iris`, so the R listings need only the three packages.

Python

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import (
    GridSearchCV,
    KFold,
    ShuffleSplit,
    StratifiedKFold,
    TimeSeriesSplit,
    cross_val_score,
    cross_validate,
    train_test_split,
)
from sklearn.neighbors import (
    KNeighborsClassifier,
)

X, y = load_iris(return_X_y=True)
knn = KNeighborsClassifier(3)
```

Holdout and Subsampling

R

```r
set.seed(3141)
i  <- sample(nrow(iris), 0.7 * nrow(iris))
tr <- iris[i, ]; te <- iris[-i, ]
m  <- knn3(Species ~ ., tr, k = 3)
mean(predict(m, te, type = "class") !=
     te$Species)
# random subsampling: 10 repeated splits
splits <- mc_cv(iris, prop = 0.7, times = 10)
err <- vapply(splits$splits, function(s) {
  m  <- knn3(Species ~ ., analysis(s), k = 3)
  te <- assessment(s)
  mean(predict(m, te, type = "class") !=
       te$Species)
}, numeric(1))
c(mean = mean(err),
  naive_se = sd(err) / sqrt(length(err)))
```

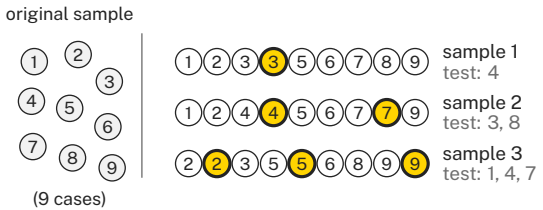
Python

```python
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.30, random_state=3141)
print(1 - knn.fit(Xtr, ytr).score(Xte, yte))
# random subsampling: 10 repeated splits
cv = ShuffleSplit(n_splits=10, test_size=0.30,
                  random_state=3141)
err = 1 - cross_val_score(knn, X, y, cv=cv)
print(err.mean(), err.std(ddof=1) / np.sqrt(10))
```

`ShuffleSplit` repeats random subsampling, as `mc_cv` does. Its test sets may overlap, so it is *not* K-fold.

The bootstrap method

1. Train a model on a random sample of size N, drawn **with replacement** from the original data set (of size N).
   - Sampling with replacement means that several cases will appear more than once on the training set.
2. Test the model on the samples that did *not* appear in the training set.
3. Repeat the process many times (typically, around 200).



Gold marks the extra copy of a case drawn more than once. Each extra copy takes the place of a case that never enters training, and those cases are the test set.

After Galdi and Tagliaferri (2019).

The .632 estimate

- On average, about **63.2%** of the distinct cases appear in the N-row training sample.
- The remaining **36.8%** of the cases (on average) can be used for testing.
- The **average score** of the repetitions is known as ϵ_0.

$$.632 \text{ estimate} = 0.368 \times \epsilon_r + 0.632 \times \epsilon_0$$

where ϵ_r is the **resubstitution** estimate.

WORKED: `iris`, 3-NN, 200 REPETITIONS

- Distinct cases per resample: **63.57%** observed, against $1 - e^{-1} = 0.6321$.
- Out-of-bag error $\epsilon_0 \approx \mathbf{0.04607}$.
- Resubstitution error $\epsilon_r = \mathbf{0.0400}$.

$$0.368(0.0400) + 0.632(0.04607)$$

$$\approx 0.01472 + 0.02912 \approx \mathbf{0.0438}$$

ϵ_0 is pessimistic (each model saw about 63% of the data), ϵ_r is optimistic. Reporting ϵ_r alone would have claimed 0.0400.

The Bootstrap

DALHOUSIE UNIVERSITY

R

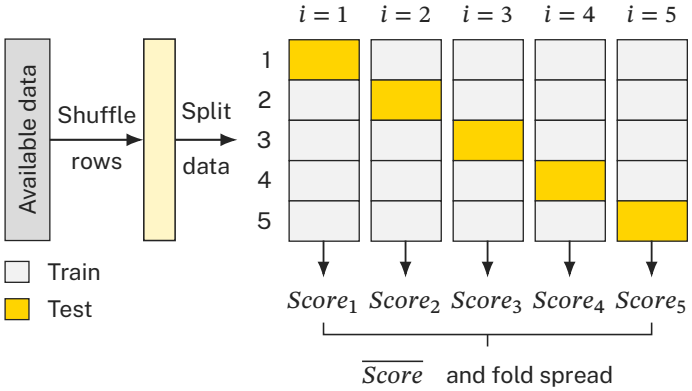
```r
oob_err <- function(d, idx) {
  tr <- d[idx, ]; te <- d[-unique(idx), ]
  m  <- knn3(Species ~ ., tr, k = 3)
  mean(predict(m, te, type = "class") !=
       te$Species)
}
set.seed(3141)
b  <- boot(iris, oob_err, R = 200)
e0 <- mean(b$t)
m  <- knn3(Species ~ ., iris, k = 3)
er <- mean(predict(m, iris, type = "class") !=
           iris$Species)
0.368 * er + 0.632 * e0           # 0.0452
trainControl()$method   # "boot" (number = 25)
```

Python

```python
rng = np.random.default_rng(3141)
n, errs = len(y), []
for _ in range(200):
    idx = rng.integers(0, n, n)
    oob = np.setdiff1d(np.arange(n), idx)
    m = KNeighborsClassifier(3).fit(
        X[idx], y[idx])
    errs.append(1 - m.score(X[oob], y[oob]))
e0 = np.mean(errs)                    # 0.0461
er = 1 - knn.fit(X, y).score(X, y)    # 0.0400
print(0.368 * er + 0.632 * e0)        # 0.0438
```

`caret::train` resamples with the *bootstrap*, 25 repetitions, unless told otherwise. `scikit-learn` has no bootstrap splitter, so the Python loop draws the indices itself.

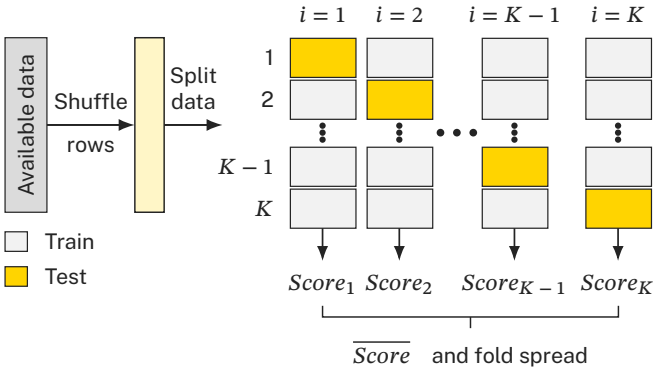
5-fold cross-validation



1. Randomly permute the rows.
2. Divide into 5 blocks of the same size (called **folds**).
3. For each fold, test a model trained on the remaining 4 folds.

Every row is tested exactly once, and trained on exactly four times.

K-fold cross-validation I



1. Randomly permute the rows.
2. Divide into K folds.
3. For each fold, test a model trained on the remaining $K - 1$ folds.

- **No overlap between test sets**, unlike random subsampling or bootstrap.

K-fold cross-validation II: choosing K

- Common K values: **5, 10, 20**.
- **Leave-one-out cross-validation** (LOOCV) if $K = N$ (N is the number of observations).
- Sometimes the whole process is repeated more than once.
  - E.g., 10×10-fold CV.

WHAT K BUYS AND COSTS

K small (5)	larger training–test mismatch, cheaper, more stable across folds
K large (20)	each model sees nearly all the data, more expensive, folds more correlated
$K = N$ (LOOCV)	no randomness left to average over; N model fits

On `iris` with 3-NN, mean errors are 0.0333 (5-fold), 0.0467 (10-fold) and 0.0400 (LOOCV). The two $s/\sqrt{K}$ values, 0.0149 and 0.0174, are naive SEs, not valid CV intervals.

Cross-Validation

R

```r
set.seed(3141)
ctrl <- trainControl(method = "cv", number = 5)
fit  <- train(Species ~ ., data = iris,
              method = "knn",
              tuneGrid = data.frame(k = 3),
              trControl = ctrl)
1 - fit$results$Accuracy          # 0.0400
```

Python

```python
cv = StratifiedKFold(n_splits=5, shuffle=True,
                     random_state=3141)
err = 1 - cross_val_score(knn, X, y, cv=cv)
print(err.mean())                    # 0.0333
# the training score beside the held-out one
res = cross_validate(knn, X, y, cv=cv,
                     return_train_score=True)
print(1 - res["train_score"].mean(),  # 0.0367
      1 - res["test_score"].mean())   # 0.0333
```

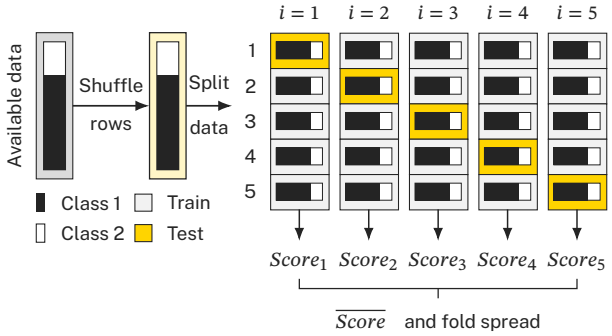
`return_train_score=True` prints the resubstitution error beside the held-out one. For 3-NN they are close, 0.0367 against 0.0333; for 1-NN the resubstitution error is zero.

Four defaults that change the answer

Decision	R	Python (scikit-learn)
Which resampler you get by default	`caret::train` resamples with the **bootstrap**, 25 repetitions. Cross-validation happens only if you write `trainControl(method = "cv")`.	`cross_val_score` uses **5-fold**, and only 5-fold.
Whether rows are shuffled	`createFolds` permutes the rows first.	Both `KFold` splitters default to `shuffle=False`. Integer `cv` stratifies a classifier; plain `KFold(5)` on a class-sorted file can drop whole classes, with no warning.
Whether folds are stratified	`createFolds` stratifies on `y`. `trainControl(index)` uses exactly the folds you pass.	Stratified for a **classifier**, plain `KFold` for a **regressor**. `cv=5` and `cv=KFold(5)` are not the same call.
Sign of the score	Error metrics are reported as errors, so smaller is better.	Scorers are written so **larger is always better**, so error metrics are negated. `"neg_mean_squared_error"` returns $-MSE$.

Set the seed in both languages, and write the splitter out in full rather than accepting `cv=5`.

Stratified K-fold CV



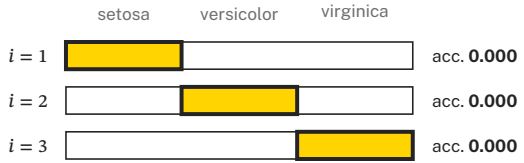


Can you see an issue if there are many more examples of one class than the other?

- In regular CV, there is a danger that each fold will not be representative of the available data.
- **Stratified CV** tries to fix this by using stratified (instead of random) sampling, ensuring that the mean response value is roughly the same across folds.
  - In binary classification: keep roughly the same relative frequency of the two classes across folds.
- However, stratifying based on the response variable compromises the "purity" of the random fold allocation.

The trap: folds cut out of a sorted file

`iris` ships sorted by species, rows 1–50 setosa, 51–100 versicolor, 101–150 virginica. Cut three folds without shuffling and each fold *is* a class.



The held-out class never appears in its own training set, so it can never be predicted. Accuracy is **zero in every fold**.

Splitter, five folds	Accuracy
`KFold(5)`, unshuffled	0.9067
`KFold(5, shuffle=True)`	0.9600
`StratifiedKFold(5, shuffle=True)`	0.9667

The 5-fold version fails quietly, six percentage points of accuracy lost with no error and no warning.

The Sorted-File Trap

R

```r
# accuracy of 3-NN, one value per held-out fold
fold_acc <- function(folds) {
  sapply(folds, function(te) {
    tr <- droplevels(iris[-te, ]) # unseen class
    m <- knn3(Species ~ ., tr, k = 3)
    p <- predict(m, iris[te, ], type = "class")
    mean(as.character(p) == iris$Species[te])
  })
}
ord <- rep(1:3, each = 50)        # file order
fold_acc(split(seq_len(nrow(iris)), ord))
# 0 0 0    <-- fold i IS species i, no warning
set.seed(3141)
f <- createFolds(iris$Species, k = 3)
mean(fold_acc(f))
# 0.947  <-- createFolds permutes AND stratifies
```

Python

```python
print(cross_val_score(knn, X, y, cv=KFold(3)))
# [0. 0. 0.]   <-- shuffle=False is the default
cv = KFold(3, shuffle=True, random_state=3141)
print(cross_val_score(knn, X, y, cv=cv).mean())
# 0.9667
```

Random generators differ across languages, so repaired accuracies need not match. Both ordered splits give zero in every fold.

One Split, Two Languages

- `KFold(3)` with `shuffle=False` cuts rows 1–50, 51–100 and 101–150, which is exactly what `rep(1:3, each = 50)` builds. The two listings are the same split.
- The zero is forced by the data, not by the seed. Each held-out block is a whole species, so that species is absent from its own training set.
- `createFolds` and integer `cv=3` stratify classification targets. The failure here explicitly requests plain `KFold(3)`.

Rules of thumb

DALHOUSIE UNIVERSITY

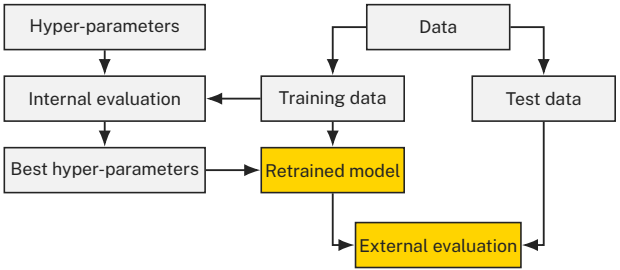
Data set	Recommended method	Why
Small (< 500 cases)	Leave-one-out CV, or the bootstrap with around 200 repetitions; the bootstrap is usually preferred	Every case is scarce. LOOCV is nearly unbiased but unstable, because its N training sets are nearly identical; the bootstrap accepts a little pessimism for a much more stable estimate
Medium (a few thousand rows)	10-fold cross-validation, often repeated several times to reduce sensitivity to one partition	Ten folds balance the pessimism of small K against the cost and correlated folds of large K
Large	Random subsampling; if the data are too large or the model too costly to refit, holdout	A single split already has enough test rows to be stable, and refitting dominates the cost

Model selection

DALHOUSIE UNIVERSITY

Choosing a model is itself a use of the data, so it needs its own evaluation, inside the first one.

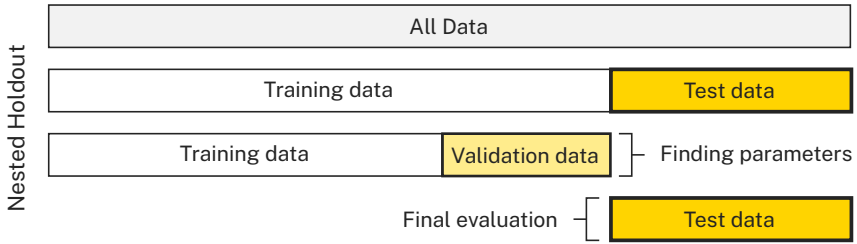
Internal and external evaluation



1. Use an **internal** evaluation method to select the best model (and/or model hyper-parameters).
2. Use an **external** evaluation method to estimate performance on unseen data.
   - Present this estimate to the end-user.

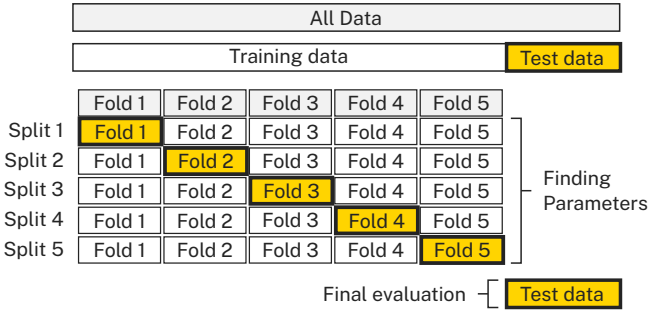
- **Nested evaluation** keeps selection inside each outer training split.

Nested holdout, drawn on the data



The test block is set aside once and touched once. Everything that involves choosing (features, hyper-parameters, thresholds, which model wins) happens strictly to the left of it.

Grid search inside cross-validation



Adapted from the scikit-learn user guide, Cross-validation.

- Selecting the best model hyper-parameters is called **hyper-parameter tuning**.
- Testing all combinations of a manually selected subset of hyper-parameters is called **grid search**.

Example on the left:

- Internal evaluation (also called **model validation**): 5-fold CV.
- External evaluation: holdout.

Nested Evaluation

R

```r
set.seed(3141)
i  <- createDataPartition(iris$Species,
        p = 0.7, list = FALSE)   # stratified
tr <- iris[i, ]; te <- iris[-i, ]
# INTERNAL: 5-fold CV over the grid, on tr only
grid <- data.frame(k = c(1, 3, 5, 9))
ctrl <- trainControl(method = "cv", number = 5)
fit  <- train(Species ~ ., data = tr,
              method = "knn", tuneGrid = grid,
              trControl = ctrl)
fit$bestTune                       # k = 9
# EXTERNAL: te is touched exactly once
mean(predict(fit, te) != te$Species)  # 0
```

Python

```python
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.30, random_state=3141,
    stratify=y)
# INTERNAL: 5-fold CV over the grid, on Xtr only
inner = StratifiedKFold(5, shuffle=True,
                        random_state=3141)
gs = GridSearchCV(KNeighborsClassifier(),
                  {"n_neighbors": [1, 3, 5, 9]},
                  cv=inner)
gs.fit(Xtr, ytr)
print(gs.best_params_)    # {'n_neighbors': 1}
# EXTERNAL: Xte is touched exactly once
print(1 - gs.score(Xte, yte))     # 0.0
```

Replacing the holdout with an outer loop gives fully nested CV. `gs.best_score_` is *internal*; never report it.

Performance estimation for time series

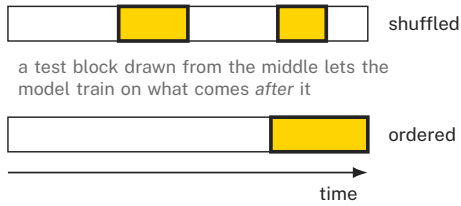
DALHOUSIE UNIVERSITY

Every method so far shuffles or resamples rows at random. Time-ordered rows cannot be shuffled.

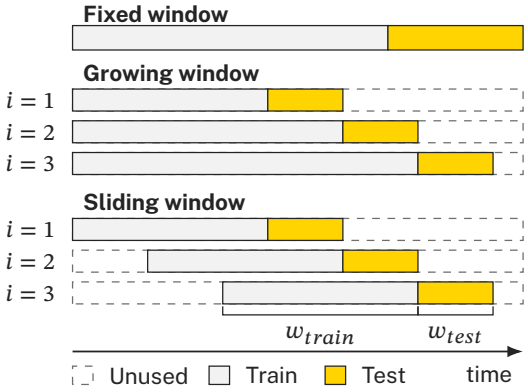
Motivation

- Most common evaluation methods rely on **randomly sampling** data to obtain an evaluation estimate for unseen data.
  - They assume there is no order (or dependence) between observations.
- In time series, there is an **inherent order** between observations (and often dependence between observations as well).

Do not ignore temporal order, and do not evaluate by training on "future" data and testing on "past" data.

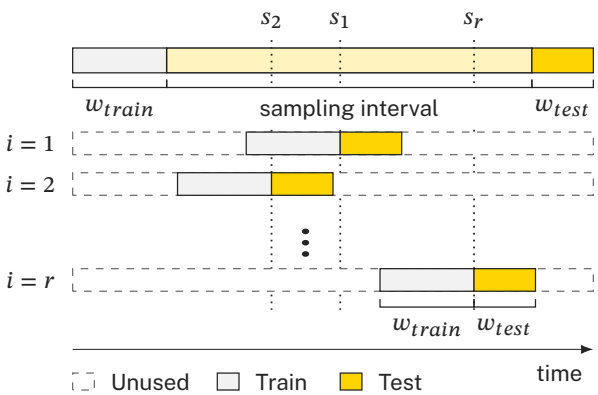


Simple train-test windows



- Divide the existing data into two temporal windows.
  - Past data (observations until a time t).
  - "Future" data (observations after t).
- **Fixed window**
  - A single model is obtained with the available "training" data, and applied to all the test period.
- **Growing window**
  - Every w_{test} test cases, obtain a new model using all data until then.
- **Sliding window**
  - Every w_{test} test cases, a new model is obtained from the previous w_{train} observations.

Monte Carlo estimates



1. Select a training window size w_{train} and a testing window size w_{test}.
2. Randomly generate r points within the interval $[w_{train}, \ldots, (n - w_{test})]$.
3. For each point, train on the previous w_{train} observations and test on the next w_{test} observations.

- If we need to tune model hyper-parameters, we can use an internal train-test window inside each training window.

Time-Ordered Estimation

R

```r
set.seed(3141)                        # a series
n <- 200; x <- cumsum(rnorm(n))
ts_df <- data.frame(x, y = 0.8 * x + rnorm(n))
w_train <- n / 2; w_test <- n / 4   # 100, 50
s_pts <- sample(w_train:(n - w_test), 10)
mse <- vapply(s_pts, function(s) {
  tr <- (s - w_train + 1):s
  te <- (s + 1):(s + w_test)
  fit <- lm(y ~ x, ts_df[tr, ])
  e <- predict(fit, ts_df[te, ]) - ts_df$y[te]
  mean(e^2)
}, numeric(1))
mean(mse)                             # 1.256
# growing window inside train()
grow <- trainControl(method = "timeslice",
  initialWindow = 100, horizon = 10,
  fixedWindow = FALSE)
```

Python

```python
rng = np.random.default_rng(3141)  # a series
x = np.cumsum(rng.normal(size=200))
Xts = x.reshape(-1, 1)
yts = 0.8 * x + rng.normal(size=200)
# growing window: train always precedes test
cv = TimeSeriesSplit(n_splits=5, test_size=10)
for tr, te in cv.split(Xts):
    assert tr.max() < te.min()
# sliding window: cap the training length
cv = TimeSeriesSplit(n_splits=5, test_size=10,
                     max_train_size=100)
scores = cross_val_score(LinearRegression(),
    Xts, yts, cv=cv,
    scoring="neg_mean_squared_error")
print(-scores.mean())   # 1.207, sign flipped
```

`TimeSeriesSplit` never shuffles; `max_train_size` makes it a sliding window.

The estimation methods

Holdout	Train on a random sample of data (without replacement) and test on the remaining observations.
Random Subsampling	Repeat holdout several times, with different random samples each time.
Bootstrap	Train on a random sample drawn *with* replacement and test on the remaining cases. Usually repeated around 200 times for small data sets.
K-fold Cross-validation (CV)	Randomly sample data into K folds and iteratively use each fold as a test set and the others as the training set. Leave-one-out CV if $K = N$ (N total observations).
Stratified K-fold CV	Use stratified sampling to create the splits and keep the mean response value roughly the same across folds.

Nesting, and time series

DALHOUSIE UNIVERSITY

NEST THE METHODS

Nest evaluation methods to select the model and/or the model hyper-parameters. The outer estimate is the one you report.

TIME SERIES ARE DIFFERENT

Common model evaluation methods cannot be used with time series. Use alternatives such as a sliding window.